Build Levels

Directed Graphs, Topological DP (Longest Path) · Mentor-Led Lab (Student Edition)

Developed by TalenciaGlobal

Difficulty ★★★ Advanced

Problem

In a DAG, the number of build stages equals the longest dependency chain (in nodes). Compute it with a topological-order dynamic program, and count the sinks (out-degree 0). You are given starter code with two functions. stages is almost complete but has a bug, and count_sinks is an empty stub. Fix the bug and implement the stub.

What to Compute

- Stages: the number of nodes on the longest dependency chain (a single node is 1 stage).
- Sinks: the number of vertices with out-degree 0.

Input / Output Format

Input: Line 1: N M. Next M lines: a directed edge u v.

Output: Two lines: Stages and Sinks.

Examples

Example 1

Input: 4 4 0 1 1 2 2 3 0 3

Output:

```
Stages: 4
Sinks: 1
```

Explanation: The longest chain 0 to 1 to 2 to 3 spans four nodes; only vertex 3 has out-degree 0.

Example 2

Input: 4 3 0 1 1 2 2 3

Output:

```
Stages: 4
Sinks: 1
```

Explanation: A straight chain of four nodes is four stages, ending at the single sink 3.

Constraints

- Stage of a node = 1 + the maximum stage among its prerequisites.
- Process vertices in topological order so each stage is final before use.
- A sink has out-degree 0.

Your Task

- Task 1 - Fix the bug. stages is flagged with a comment, but the bug's location is not given. Find the single bug so each dependency step adds one stage (a successor is its predecessor's stage PLUS ONE).
- Task 2 - Implement count_sinks from scratch: the number of vertices with out-degree 0.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
import sys
import heapq
from collections import deque

def read_ints():
    return [int(x) for x in sys.stdin.read().split()]

def build_directed(data):
    n = data[0]
    m = data[1]
    adj = [set() for _ in range(n)]
    indeg = [0] * n
    idx = 2
    for _ in range(m):
        u = data[idx]
        v = data[idx + 1]
        idx += 2
        if v not in adj[u]:
            adj[u].add(v)
            indeg[v] += 1
    adj = [sorted(s) for s in adj]
    return n, m, adj, indeg, idx

def topo_order(n, adj, indeg):
    deg = indeg[:]
    heap = [v for v in range(n) if deg[v] == 0]
```

```
    heapq.heapify(heap)
    order = []
    while heap:
        u = heapq.heappop(heap)
        order.append(u)
        for w in adj[u]:
            deg[w] -= 1
            if deg[w] == 0:
                heapq.heappush(heap, w)
    return order

# BUG: this function contains a single bug. Each step along a dependency chain must
# add one stage, so a successor's level is its predecessor's level PLUS ONE.
def stages(n, adj, indeg):
    order = topo_order(n, adj, indeg)
    level = [1] * n
    for u in order:
        for w in adj[u]:
            if level[u] > level[w]:
                level[w] = level[u]
    return max(level) if n > 0 else 0

# TODO: implement this. It must count the sink vertices (out-degree 0).
# Currently a stub.
def count_sinks(n, adj):
    return 0

def main():
    data = read_ints()
    n, m, adj, indeg, idx = build_directed(data)
    print("Stages: " + str(stages(n, adj, indeg)))
    print("Sinks: " + str(count_sinks(n, adj)))

main()
```

C++

```
// Lab 6 - Build Levels (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <set>
#include <queue>
#include <algorithm>
using namespace std;

vector<long long> readInts() {
    vector<long long> data;
    long long x;
    while (cin >> x) {
        data.push_back(x);
    }
    return data;
}

vector<vector<int>> buildDirected(const vector<long long>& data, int& n, int& m,
vector<int>& indeg, int& idx) {
```

```
    n = (int) data[0];
    m = (int) data[1];
    vector<set<int>> tmp(n);
    indeg.assign(n, 0);
    idx = 2;
    for (int i = 0; i < m; i++) {
        int u = (int) data[idx];
        int v = (int) data[idx + 1];
        idx += 2;
        if (tmp[u].find(v) == tmp[u].end()) {
            tmp[u].insert(v);
            indeg[v]++;
        }
    }
    vector<vector<int>> adj(n);
    for (int i = 0; i < n; i++) {
        adj[i] = vector<int>(tmp[i].begin(), tmp[i].end());
    }
    return adj;
}

vector<int> topoOrder(int n, const vector<vector<int>>& adj, const vector<int>& indeg) {
    vector<int> deg = indeg;
    priority_queue<int, vector<int>, greater<int>> heap;
    for (int v = 0; v < n; v++) {
        if (deg[v] == 0) {
            heap.push(v);
        }
    }
    vector<int> order;
    while (!heap.empty()) {
        int u = heap.top();
        heap.pop();
        order.push_back(u);
        for (int w : adj[u]) {
            deg[w]--;
            if (deg[w] == 0) {
                heap.push(w);
            }
        }
    }
    return order;
}

// BUG: this function contains a single bug. Each step along a dependency chain must
// add one stage, so a successor's level is its predecessor's level PLUS ONE.
int stages(int n, const vector<vector<int>>& adj, const vector<int>& indeg) {
    vector<int> order = topoOrder(n, adj, indeg);
    vector<int> level(n, 1);
    for (int u : order) {
        for (int w : adj[u]) {
            if (level[u] > level[w]) {
                level[w] = level[u];
            }
        }
    }
    int best = 0;
    for (int v = 0; v < n; v++) {
```

```
        if (level[v] > best) {
            best = level[v];
        }
    }
    return best;
}

// TODO: implement this. It must count the sink vertices (out-degree 0).
// Currently a stub.
int countSinks(int n, const vector<vector<int>>& adj) {
    return 0;
}

int main() {
    vector<long long> data = readInts();
    int n, m, idx;
    vector<int> indeg;
    vector<vector<int>> adj = buildDirected(data, n, m, indeg, idx);
    cout << "Stages: " << stages(n, adj, indeg) << "\n";
    cout << "Sinks: " << countSinks(n, adj) << "\n";
    return 0;
}
```

Java

```
// Lab 6 - Build Levels (STARTER)
import java.util.*;

public class Main {
static int N, M;
    static List<Long> DATA;
    static int EDGE_END;
    static int[] INDEG;

    static List<List<Integer>> buildDirected(Scanner sc) {
        List<Long> data = new ArrayList<>();
        while (sc.hasNextLong()) {
            data.add(sc.nextLong());
        }
        DATA = data;
        N = (int) (long) data.get(0);
        M = (int) (long) data.get(1);
        List<TreeSet<Integer>> tmp = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            tmp.add(new TreeSet<>());
        }
        INDEG = new int[N];
        int idx = 2;
        for (int i = 0; i < M; i++) {
            int u = (int) (long) data.get(idx);
            int v = (int) (long) data.get(idx + 1);
            idx += 2;
            if (!tmp.get(u).contains(v)) {
                tmp.get(u).add(v);
                INDEG[v]++;
            }
        }
```

```
    EDGE_END = idx;
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < N; i++) {
        adj.add(new ArrayList<>(tmp.get(i)));
    }
    return adj;
}

static List<Integer> topoOrder(int n, List<List<Integer>> adj, int[] indeg) {
    int[] deg = indeg.clone();
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int v = 0; v < n; v++) {
        if (deg[v] == 0) {
            heap.add(v);
        }
    }
    List<Integer> order = new ArrayList<>();
    while (!heap.isEmpty()) {
        int u = heap.poll();
        order.add(u);
        for (int w : adj.get(u)) {
            deg[w]--;
            if (deg[w] == 0) {
                heap.add(w);
            }
        }
    }
    return order;
}

// BUG: this function contains a single bug. Each step along a dependency chain must
// add one stage, so a successor's level is its predecessor's level PLUS ONE.
static int stages(int n, List<List<Integer>> adj, int[] indeg) {
    List<Integer> order = topoOrder(n, adj, indeg);
    int[] level = new int[n];
    Arrays.fill(level, 1);
    for (int u : order) {
        for (int w : adj.get(u)) {
            if (level[u] > level[w]) {
                level[w] = level[u];
            }
        }
    }
    int best = 0;
    for (int v = 0; v < n; v++) {
        if (level[v] > best) {
            best = level[v];
        }
    }
    return best;
}

// TODO: implement this. It must count the sink vertices (out-degree 0).
// Currently a stub.
static int countSinks(int n, List<List<Integer>> adj) {
    return 0;
}
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<List<Integer>> adj = buildDirected(sc);
        System.out.println("Stages: " + stages(N, adj, INDEG));
        System.out.println("Sinks: " + countSinks(N, adj));
    }
}
```

Sample Run

Sample Input

```
4 4
0 1
1 2
2 3
0 3
```

Sample Output

```
Stages: 4
Sinks: 1
```